

METRICS.md — HW2.5

GPU: NVIDIA GeForce RTX 4090 UUID: GPU-5b052ad1-4272-40db-4b25-c930bf32b547 Driver: 595.95, CUDA 13.2 (see `rtx4090_provenance_auto.json`, `rtx4090_nvidia_smi_q.txt`) Vendor specs (NVIDIA Ada GPU Architecture whitepaper, <https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf>): Ada Lovelace / AD102, TSMC 4N, GDDR6X 384-bit, ~1008 GB/s peak bandwidth, 512 4th-gen Tensor Cores (FP16/BF16/TF32/INT8/FP8 support), 82.6 TFLOPS FP32, 450 W TDP.

Table HW2.5.1 — Summary

Measurement	Your GPU	Notes
Peak achieved TFLOPS (BF16)	164.37	N=8192, 30 reps; RUN_LOG.txt line for n=8192/BF16
% of theoretical peak (BF16)	~99.6% of dense FP16 tensor peak (165 TFLOPS per whitepaper spec sheet)	see note below on peak reference used
Effective bandwidth (GB/s)	910.63 (90.3% of ~1008 GB/s spec)	elementwise copy, 268,435,456 elements, RUN_LOG.txt Part C
Naive attention OOM length	largest OK = 26832, smallest OOM = 26833	with <code>--mem-fraction 0.9</code> ; see Part D note
Fused attention OOM length	not reached up to S=32768 (uncapped)	see Part D note
Steady-state / peak throughput	100.0% (48.457 / 48.467 calls/s)	matmul N=8192, 20 min sustained
Throttle onset (s, or none)	~5s (power-limit, not thermal)	see Part E note

All values above are traceable to lines in `RUN_LOG.txt` and the CSVs listed per-row.

Part A — Provenance

Captured via `part_a_provenance.py` from live `nvidia-smi -q` output (`rtx4090_nvidia_smi_q.txt`).

UUID, driver, CUDA version, VRAM, and power limit are auto-parsed into

`rtx4090_provenance_auto.json`. Vendor-doc fields (architecture, memory type/bandwidth, tensor core generation, supported precisions) are cited above from NVIDIA's own Ada GPU Architecture whitepaper.

Part B — Precision and achieved throughput

30 repetitions per (N, precision) configuration, timed with CUDA events after a 10-call warmup.

N	FP32	TF32	FP16	BF16
1024	44.72	65.14	98.46	120.07
4096	52.98	79.03	167.09	157.91
8192	52.31	86.41	162.89	164.37
16384	50.25	86.95	157.91	160.57

(TFLOPS; see `rtx4090_part_b_throughput.csv` and `rtx4090_part_b_throughput.png`)

Plateau behavior. FP32 and TF32 are already near their plateau by N=4096 and stay flat (within ~5%) through N=16384 — FP32 doesn't use tensor cores on Ada, so its ceiling is the CUDA-core FMA rate, reached once N is large enough to keep the SMs fully occupied. FP16/BF16 (tensor core paths) actually peak earliest at N=4096 (167.09/157.91) and settle slightly lower at N=8192-16384 (~160-165) — small matrices (N=1024) never reach the plateau because the matmul is too short to amortize kernel launch overhead and fill all SMs; at N=1024 tensor-core throughput is still only ~73-75% of its $N \geq 4096$ value.

Note on FP32 vs. spec: achieved FP32 (~50-53 TFLOPS) sits well below the 82.6 TFLOPS FP32 spec peak — this is expected, since that spec figure assumes ideal occupancy conditions rarely reached by a single unfused matmul call at these sizes; the ~60-64% peak-utilization figure is reasonable for measured throughput.

Part C — Bandwidth-bound vs. compute-bound

- **Memory-bound (elementwise copy)**, 268,435,456 FP32 elements (1 GiB/buffer): 910.63 GB/s effective, **90.3%** of the ~1008 GB/s spec bandwidth. Arithmetic intensity = 0 FLOPs/byte (copy does no floating-point work) — the textbook memory-bound extreme.
- **Compute-bound (square matmul, N=8192, FP32)**: 54.54 achieved TFLOPS, arithmetic intensity = 1365.33 FLOPs/byte (= $N/6$ for a naive-bytes-touched accounting of one read of A,

one read of B, one write of C).

Roofline classification. Ridge point $\approx \text{peak_TFLOPS} / \text{peak_bandwidth} = 165\text{e}12 / 1008\text{e}9 \approx 164$ FLOPs/byte (using measured BF16 peak as the compute ceiling). The matmul's AI (1365) is roughly 8x past the ridge — solidly compute-bound, consistent with it reaching near-peak TFLOPS in Part B. The copy sits at AI=0, deep on the memory-bound side, and achieves 90% of peak bandwidth — close to the practical ceiling for a simple copy kernel.

Part D — The cost of attention

Config: batch=1, heads=8, head_dim=64, dtype=float16.

S	naive latency	naive peak mem	fused latency	fused peak mem
512	0.08-0.21 ms	0.02 GB	0.03-0.04 ms	0.01 GB
1024	0.07-0.18 ms	0.05 GB	0.05-0.08 ms	0.01 GB
2048	0.40 ms	0.15 GB	0.10 ms	0.02 GB
4096	1.86-1.93 ms	0.56 GB	0.30 ms	0.03 GB
8192	7.21-7.23 ms	2.19 GB	1.11-1.17 ms	0.04 GB
16384	28.65-28.68 ms	8.67 GB	4.44-4.48 ms	0.08 GB
20480	44.35-44.43 ms	13.51 GB	6.66-6.84 ms	0.09 GB
24576	64.89-65.01 ms	19.44 GB	9.85-10.00 ms	0.11 GB
28672	325.23 ms (uncapped run) / OOM (mem-fraction 0.9 run)	26.43 GB (uncapped)	13.55-13.68 ms	0.13 GB
32768	1172.97 ms (uncapped, WDDM spillover)	34.50 GB (uncapped, exceeds physical VRAM)	17.82-17.91 ms	0.14 GB

OOM boundary — naive: largest OK = 26832, smallest OOM = 26833 (single-token resolution via binary search).

Note on methodology: on this Windows/WDDM system, an uncapped run never raised `torch.cuda.OutOfMemoryError` even past 34.5 GB reported peak allocation on a 24 GB card — the latency jump at $S=28672 \rightarrow 32768$ (65ms $\rightarrow$ 325ms $\rightarrow$ 1173ms, a 5x then 18x slowdown) is WDDM's driver-level "shared GPU memory" fallback silently spilling allocations into system RAM over PCIe rather than raising CUDA OOM. To recover a genuine, physical-VRAM-bound OOM boundary as the assignment specifies, the CUDA allocator was capped to 90% of VRAM (`torch.cuda.set_per_process_memory_fraction(0.9)`), which produced the clean 26832/26833 boundary above.

OOM boundary — fused: not reached up to $S=32768$, the largest tested. Fused (SDPA/FlashAttention) memory scales close to linearly with sequence length (0.01 GB at $S=512 \rightarrow$ 0.14 GB at $S=32768$, a 64x increase in S for a 14x increase in memory) rather than quadratically, so it simply doesn't hit a VRAM wall in the range tested here.

Quadratic memory fit (naive): $\text{peak_memory_GB} \approx 3.2\text{e-}08 \times S^2 + \dots$ (see `rtx4090_part_d_memory.png`). Theory: the dominant term comes from materializing the (B,H,S,S) scores/weights tensors in fp16 — with $B=1$, $H=8$, 2 bytes/element, one such tensor alone contributes $B \times H \times 2 / 1\text{e}9 = 1.6\text{e-}08$ GB/token². The measured coefficient (3.2e-08) is almost exactly 2x that, consistent with the naive implementation holding both the pre-softmax `scores` tensor and the post-softmax `weights` tensor as separate full (B,H,S,S) allocations simultaneously.

Speedup (fused vs. naive): ~6.4x at $S=16384$ (4.46ms vs 28.65ms), ~6.5x at $S=24576$ (10.00ms vs 64.89ms) — consistent across scale.

What the fused kernel avoids: FlashAttention-style fused SDPA never materializes the full (S,S) score or weight matrix in global memory. It tiles the computation, keeping blocks of Q/K/V and partial softmax statistics in fast on-chip SRAM, and accumulates the output incrementally across tiles using an online-softmax algorithm. This avoids both the $O(S^2)$ memory footprint (peak memory stays effectively $O(S)$ here) and the extra global-memory read/write traffic naive attention pays for writing out and re-reading the full attention matrix, which is why it is also several times faster despite doing asymptotically the same FLOPs.

Part E — Sustained load and thermal behaviour

20-minute sustained load, matmul $N=8192$ FP32, sampled every 5s (`rtx4090_thermal_log.csv`, `rtx4090_part_e_thermal.png`).

- **Temperature:** climbed from 43°C (idle) to ~65-76°C within the first ~100s, then held flat at 75-76°C for the remainder of the run — no continued climb.

- **Power:** reached the 450 W cap within the first 5s sample and stayed pegged at 448-450 W for the entire run.
- **Clock:** oscillated narrowly between 2175-2250 MHz throughout, hunting around the power ceiling rather than showing a sustained downward ramp.
- **Throttle onset:** effectively immediate (~5s) — this is **power-limit throttling**, not thermal throttling: temperature plateaued at 75-76°C, well below the ~90°C thermal limit typical for this card, while power sat at its 450W TDP the entire time.
- **Throughput retention:** peak (first 30s) = 48.467 calls/s, steady-state (final 5 min) = 48.457 calls/s → **100.0%** retention. The fine-grained clock oscillation (a few dozen MHz around the power ceiling) produced no measurable throughput loss between the "cold" and steady-state windows — the power ceiling was reached almost immediately and throughput was stable from then on, rather than showing a classic thermal-ramp degradation curve.

AI use

See `AI_USE.md`.